

On-device ML for IMU based gesture recognition as an activating trigger for a security monitor

Florian Bohnes
252061IV

I. INTRODUCTION

This project presents a comprehensive embedded gesture recognition pipeline, encompassing data acquisition, preprocessing, model training, benchmarking, and final deployment on a Raspberry Pi Pico microcontroller. The final deployed model is able to classify IMU signals as predefined gestures and uses these to trigger the start of a server-side application for security monitoring. The architecture design for a neural network model was informed by a review of some relevant literature. This review also inspired an exploratory investigation into the use of Generative Adversarial Networks (GANs) for creating synthetic data to improve the stability of the training pipeline. This is discussed in section II-B but was not incorporated into the final system.

II. STATE OF THE ART

To inform the methodology adopted in this project, two state-of-the-art approaches were investigated:

- 1) established neural network architectures for gesture classification using IMU data [1], [2]
- 2) a GAN-based method for synthetic training data augmentation [3].

A. System Design

[1] provides a detailed overview of a deep learning pipeline for the classification of hand gestures, which has been proven to reach better classification metrics than traditional machine learning methods, such as Support Vector machines and k-NN classifiers. The demonstrated method employs feature analysis as well as dimensionality reduction using Principal Component Analysis. The authors go as far as implementing a Deep Neural Network (DNN) for classification and reach a 3-5 % accuracy improvement over traditional methods. However, this approach is unsuitable for the project due to hardware restrictions. System design aims to find an economical solution. 1D Convolutional Neural Networks (CNNs) are a natural fit for inertial measurement-based gesture recognition because their architecture aligns well with the usage of multi-channel time series data. For example, one study titled “Gesture Recognition Using Inertial Sensors with 1D Convolutional Neural Network” [2] reports achieving 89.6% accuracy on a 10-class Arabic-numeral gesture set using 3-axis accelerometer + 3-axis gyroscope data and a 1D CNN architecture. Instead of manually extracting features, this model learns features directly from raw data. This study has strongly inspired system design in the project.

B. Synthetic Data: Investigations into using GAN for this project

One of the principal challenges in machine learning for gesture recognition using IMU data is the general scarcity of labeled sensor data. Collecting sufficiently large datasets is a time-consuming and exhaustive process. Augmenting the dataset through generative AI is a promising approach to alleviating this bottleneck. [3] demonstrates such an approach using GANs. They design a cGAN named TheraGAN to generate 6-channel IMU (linear acceleration + angular velocity) signals for rehabilitation activities. The authors demonstrate that incorporating synthetic data into the training set of deep classifiers enhances classification performance. In their experiments, the best improvement was about 13.27%.

III. DESCRIPTION OF THE METHODOLOGY

A. Full System Architecture

The complete system architecture follows a modular two-application paradigm designed to separate time-critical gesture recognition from higher-level communication and server interaction. This separation is crucial for achieving reliable real-time performance on constrained hardware, while still enabling networked functionality and external processing. The overall architecture is illustrated in Fig. 1 and spans offline model training, on-device inference, and optional server interaction. This figure also includes the GAN-based augmentation that has later been excluded from the workflow (Sec. III-D2).

B. Dual-Application Paradigm

To balance low-latency gesture recognition with intermittent high-bandwidth data transmission, the system is structured around a dual application paradigm running on the same embedded device. The core idea is to decouple continuous sensing and inference from network-dependent streaming. A lightweight, always-on application performs local gesture classification, while a secondary application responsible for network communication is activated only when a relevant event is detected.

This design minimizes unnecessary wireless activity and computational load during normal operation, while still enabling rich data capture when a gesture of interest occurs. The transition between applications is event-driven: a specific classifier output (e.g. a ring gesture) acts as a trigger that temporarily switches the system into a streaming mode.

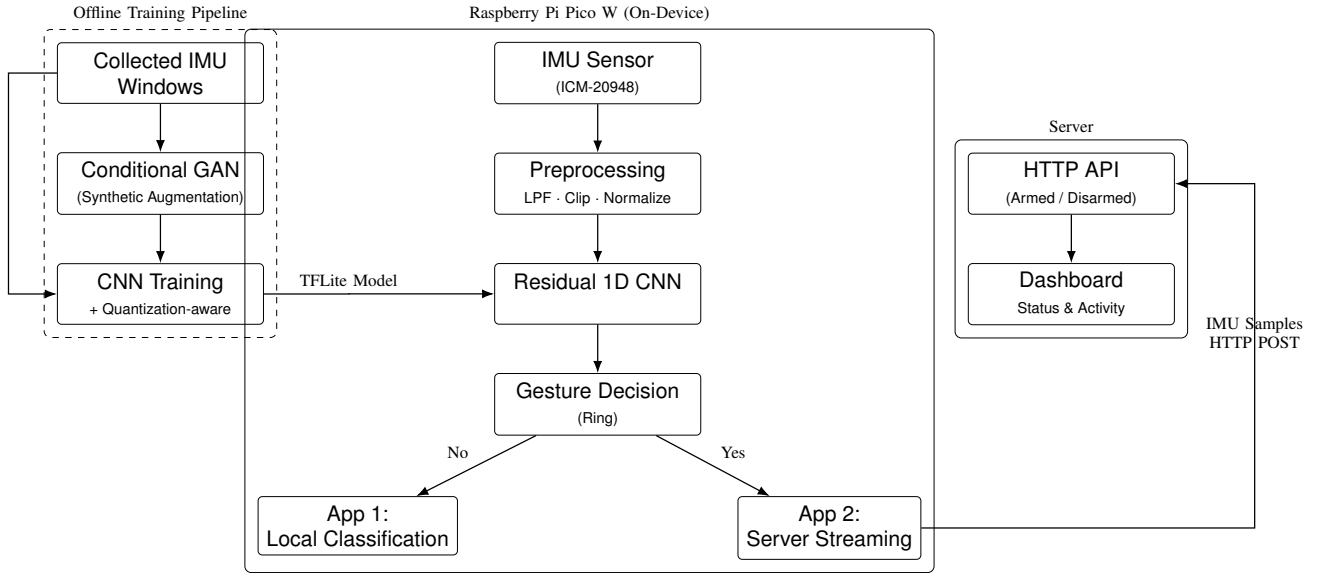


Fig. 1. Full System Architecture, including training pipeline with augmentation, on-device preprocessing and classification, and server-side application

C. Network Architecture

The final gesture classification model is implemented as a fully convolutional 1D network operating on fixed-length IMU windows of shape $(T = 64, C = 3)$. In contrast to [2] the model relies exclusively on temporal convolutions. This enables full compatibility with TensorFlow Lite.

Per-channel normalization is followed by a chain of residual blocks. These blocks combine two Conv1D layers with batch normalization and LeakyReLU activations. Temporal resolution is progressively reduced using max-pooling, while the number of feature channels is increased to capture more abstract motion patterns. SpatialDropout1D is applied after pooling stages to regularize feature maps without disrupting temporal structure.

To increase the temporal receptive field without additional downsampling, a dilated convolutional block is introduced in the later stage of the network. This allows the model to integrate longer-range temporal dependencies while maintaining a compact representation. The resulting feature maps are aggregated using global average pooling, eliminating the time dimension and yielding a fixed-size embedding independent of the input window length.

The classification head consists of a dense layer with ReLU activation and batch normalization, followed by optional dropout and a final softmax layer producing class probabilities. Overall, the architecture balances representational capacity and computational efficiency, providing a TFLite-friendly alternative to CNN-Recurrent Neural Network (RNN) hybrids while retaining strong temporal modeling capabilities.

D. Considered Approach for Data Augmentation

1) *GAN training in PyTorch within a TensorFlow-based project:* The main gesture-recognition pipeline of this project is implemented in TensorFlow/Keras [4] in order to align with

the downstream deployment target (TensorFlow Lite) and to maintain a single, consistent toolchain for model training, export, and embedded inference. However, GAN-based data augmentation was treated as an exploratory investigation with a strong emphasis on rapid iteration and debugging of training dynamics.

PyTorch [5] was therefore chosen specifically for GAN training. In practice, GAN optimization requires alternating updates of two networks, flexible loss definitions, and frequent inspection of intermediate tensors and gradients to stabilize training (e.g., when diagnosing mode collapse, gradient explosion, or discriminator overpowering). PyTorch's eager-by-default execution model and lightweight training-loop ergonomics enabled faster experimentation with such stabilizing techniques, including WGAN-GP objectives, discriminator regularization, and additional auxiliary losses, without coupling these experiments to the TensorFlow deployment stack.

Importantly, this choice does not imply a limitation of TensorFlow in implementing GANs; rather, it reflects an engineering trade-off between (i) a stable, deployment-oriented TensorFlow pipeline for the final system and (ii) a research-oriented PyTorch workflow for exploratory GAN training. The synthetic samples produced by the GAN were exported in the same file format as the original dataset, allowing them to be consumed by the existing TensorFlow training pipeline without modifying the final system architecture.

2) *Exclusion of GAN-based augmentation from the final system:* Despite achieving stable training dynamics and generating some visually plausible IMU sequences, GAN augmentation was ultimately excluded from the final system due to computational constraints. Relative to the scope of the project, the approach for synthesization was found to be too ambitious. Training times are quickly disproportionate to the marginal performance gains observed. This cost was further amplified

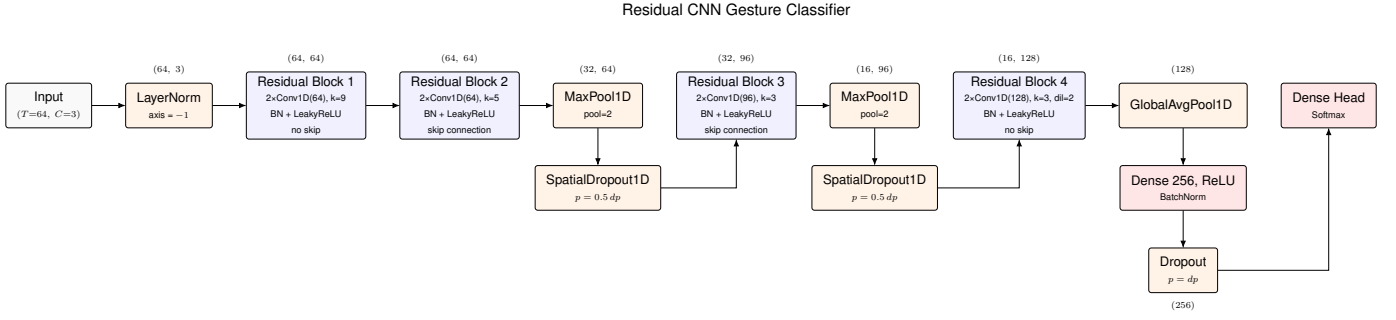


Fig. 2. Full Network Architecture

by the need for repeated training runs when tuning hyper-parameters to maintain stability across gesture categories. Because the final gesture recognition system targets resource-constrained deployment, the inclusion of a computationally expensive data generation stage was difficult to justify in the absence of consistent and significant improvements over simpler augmentation methods. Given these considerations, the project prioritized approaches that offered a more favorable trade-off between implementation complexity, computational cost, and empirical benefit. Consequently, GAN-based augmentation was treated as an exploratory investigation rather than a core component of the final system. The findings nonetheless provide useful insight into the limitations of adversarial sequence generation for small-scale IMU datasets and inform potential future work under less restrictive computational constraints.

IV. IMPLEMENTATION

A. Software Requirements

Variably, different technology stacks were required in different subsets of the project. Different requirements of Python libraries were necessary for:

- Training the model
- Training the GAN
- Setting up and running the Server

For reproducibility, `environment.yml` files can be found in the relevant part of the project structure. This facilitates creation of conda environments to recreate results.

```
gesture_recog/
  environment.yml
  gan/
    environment.yml
server/
  environment.yml
```

B. Preprocessing Pipeline

To ensure consistency between data collection and inference, the following preprocessing steps are applied to the raw IMU data:

- 1) **Low-pass filtering:** A low-pass filter is applied to smooth the data and remove high-frequency noise.

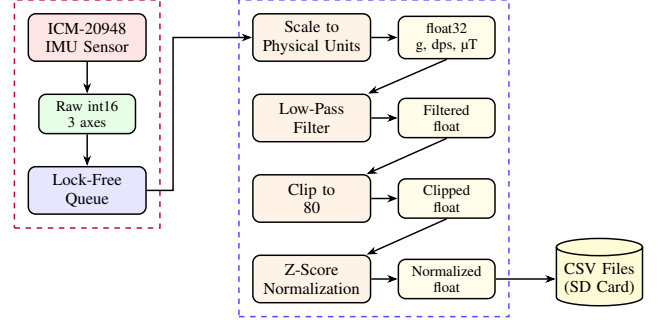


Fig. 3. Preprocessing Pipeline

- 2) **Normalization:** The data is normalized to a fixed range (e.g., $[-1, 1]$) using predefined scaling factors for each sensor axis.
- 3) **Clipping:** The data is clipped to a range of $[-80, 80]$ to handle outliers.
- 4) **Z-score normalization:** Each axis is normalized using Z-score normalization to ensure zero mean and unit variance.

Figure 3 illustrates the preprocessing pipeline, leading into data storage IV-C. The steps of the pipeline are replicated later in inference IV-D.

C. Data Storage

Recorded data is written to an SD card with a FAT32 filesystem. Recordings are saved as separate gesture session samples with the following structure:

- Header:


```
sample <SESSION> \n ax,ay,az \n
```
- Data rows:


```
<ax>, <ay>, <az> \n
```
- Separators: 4 empty lines between sessions

Each gesture is recorded in sequence, with all samples for a gesture collected before moving to the next gesture. The session number is incremented for each sample, ensuring unique identifiers for all data.

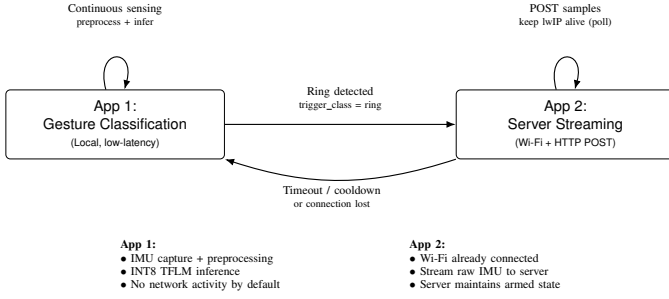


Fig. 4. Dual Application Paradigm

D. Inference

Inference is executed locally on the Pico W using an INT8 TensorFlow Lite Micro model within a deterministic state-driven control loop. IMU acceleration data are sampled at 100 Hz and buffered for a fixed recording duration. Raw sensor values are converted to physical units to match the training data representation (Sec. IV-B). Leading idle samples below a motion threshold are excluded from the recording window, which then undergoes lightweight preprocessing, identical to the preprocessing pipeline. The resulting features are quantized to INT8 and copied directly into the model input tensor. Inference is performed synchronously via the TFLM interpreter. This design avoids dynamic memory allocation, recurrent layers, and network dependencies during inference, ensuring predictable latency and compatibility with resource-constrained deployment.

E. Dual-App Paradigm

In the implemented system, App 1 continuously captures IMU data, applies preprocessing, and performs low-latency inference using an INT8 TensorFlow Lite Micro model. It operates entirely locally and does not initiate any network communication by default, ensuring predictable timing and low power consumption.

When App 1 detects a predefined trigger class, control is handed over to App 2. App 2 assumes an already established Wi-Fi connection and streams raw IMU samples to a remote server using HTTP POST requests, keeping the network stack active via polling. The start of this streaming process changes an internal server variable from unarmed to armed. It will now analyze statistical features of the streamed IMU data to detect movement. Detection of such movement beyond defined thresholds will lead to a warning. Figure 4 illustrates the approach.

F. Hardware

The Hardware used for this project is a *Raspberry Pi Pico* Microcontroller. Further, an ICM-20948 IMU was chosen for data collection. This makes accelerometer, gyroscope, and magnetometer data available for analysis. Recorded data is written to an SD card with a FAT32 filesystem.

1) *Dual Core Architecture*: *Core 0* is dedicated to real-time sensor data acquisition at a configurable data rate. This minimizes processing overhead to maintain a consistent sampling rate. Timestamped samples are pushed into a queue. *Core 1* handles the preprocessing and SD card writes. Keeping all SD activity on one core avoids concurrency bugs and the need for locks. This also circumvents a firmware bug in the Pico, where SD writing from *Core 1* has not been possible so far. Sampling is run through a queue, which means it can run at a steady rate even when SD writes are slow. Placing the tight, time-sensitive sampling loop on one core reduces jitter caused by SD I/O.

V. RESULTS AND ANALYSIS

A. Classification Performance

The gesture classification performance was evaluated on a held-out test set using standard multiclass metrics. Table I summarizes the overall accuracy and macro-averaged scores for the final deployed model.

TABLE I
OVERALL CLASSIFICATION PERFORMANCE

Model Variant	Accuracy [%]	Precision	Recall	F1-score
Float32 (offline)	93.4	0.934	0.931	0.932
INT8 (TFLite Micro)	92.1	0.921	0.918	0.919

Quantization to INT8 resulted in a modest accuracy degradation of approximately 1.3 percentage points, while preserving the overall class ranking and confusion structure. Although model development prioritized classification accuracy, inference latency on the target device was measured to ensure real-time feasibility. The deployed INT8 model achieves sub-10 ms inference time, satisfying the timing constraints imposed by continuous IMU sampling.

B. Per-Class Performance

Table II reports per-class precision and recall for the deployed INT8 model.

TABLE II
PER-CLASS PERFORMANCE (INT8 MODEL)

Class	Precision	Recall
Negative / Idle	0.95	0.97
Ring	0.94	0.88
Slope	0.88	0.90
Wing	0.91	0.91

C. Memory Footprint

The memory footprint of the deployed firmware was analyzed using the linked ELF binary to obtain an accurate breakdown of flash and RAM usage. Only loadable sections relevant to execution on the target device are reported.

The dominant contributors to flash usage are the embedded INT8 TensorFlow Lite Micro model (approximately 281 kB) and additional read-only constant data, while static RAM usage is primarily driven by the TensorFlow Lite Micro tensor

TABLE III
MEMORY USAGE ON THE TARGET DEVICE

Component	Section	Size
Application code	.text	535 kB
Read-only data (model, constants)	.rodata	589 kB
Initialized data	.data	11 kB
Static RAM (zero-initialized)	.bss	113 kB
Total flash usage		1.14 MB
Static RAM usage		113 kB

arena (80 kB) and application-level inference buffers. Debug sections in the ELF binary are excluded from these figures because they are not flashed to the device.

VI. CONCLUSIONS AND FUTURE RESEARCH

This project demonstrates that accurate, real-time gesture classification from IMU data can be achieved on a resource-constrained embedded platform using a carefully designed convolutional neural network. By selecting a fully convolutional architecture, the system achieves high classification accuracies. The separation of local gesture inference from network communication through a dual-application architecture enables low-latency, deterministic operation while preserving the ability to stream raw sensor data on demand.

Several directions for future research and development can be identified. Model generalization can be improved by collecting data from a larger, more diverse set of users and operating conditions. Data augmentation has also been shown to be promising (Sec. II-B), but was not implemented here.

ACRONYMS

CNN Convolutional Neural Network. 1, 2

DNN Deep Neural Network. 1

GAN Generative Adversarial Network. 1

IMU Inertial Measurement Unit. 1–5

RNN Recurrent Neural Network. 2

REFERENCES

- [1] K. Suri and R. Gupta, “Classification of hand gestures from wearable imus using deep neural network,” in *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. IEEE, 2018, pp. 45–50.
- [2] Z. He, Q. Li, and Z. He, “Gesture recognition using inertial sensors with 1d convolutional neural network,” in *2023 IEEE International Conference on Control, Electronics and Computer Technology (ICCECT)*. IEEE, 2023, pp. 1456–1460.
- [3] M. Mohammadzadeh, A. Ghadami, A. Taheri, and S. Behzadipour, “cgan-based high dimensional imu sensor data generation for enhanced human activity recognition in therapeutic activities,” *Biomedical Signal Processing and Control*, vol. 103, p. 107476, 2025.
- [4] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [5] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, “Automatic differentiation in pytorch,” in *NIPS-W*, 2017.